

Auditing your model for subgroup deployment readiness

Discrimination, calibration and clinical utility — and how to run it with `isitfair`

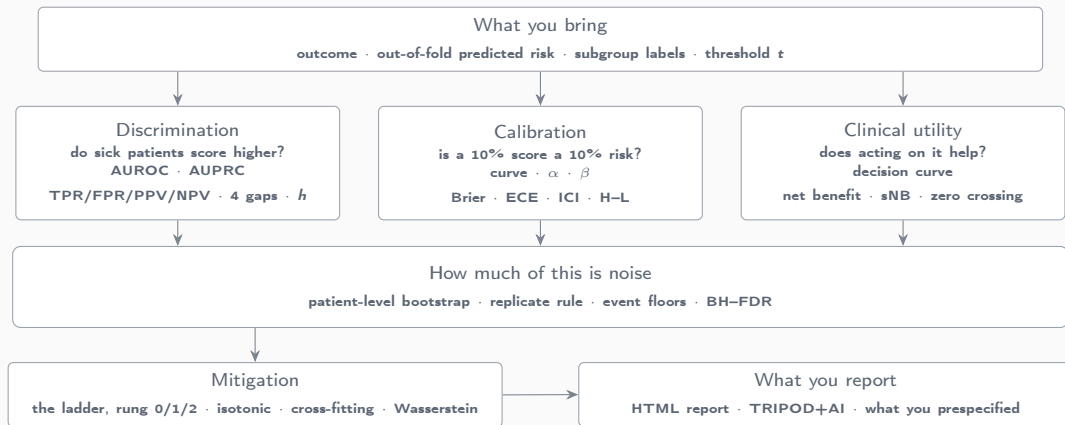
Hugo Guillen-Ramirez

August 30, 2026

Where we are going

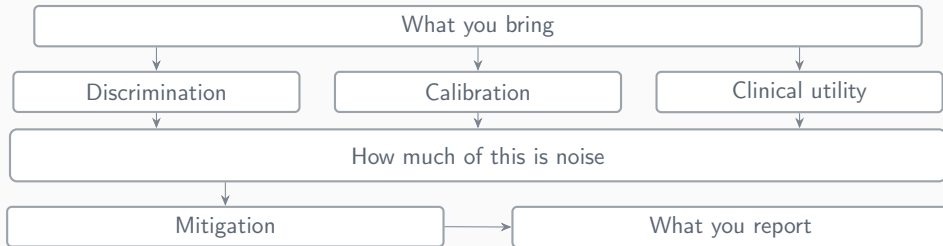
1. Why audit at all
2. The three axes
3. Setting up your audit
4. Discrimination
5. Calibration
6. Clinical utility
7. Uncertainty, floors and multiplicity
8. Mitigation
9. Reporting, and the whole thing at once

The whole audit on one page



Read down. Every section page repeats this map with the current step marked.

Why audit at all



A model that looks finished

A deployed end-of-surgery model predicting postoperative bacterial infection. 10,719 procedures, 504 events. The headline number:

$$\text{AUROC} = 0.876 \quad (95\% \text{ CI } 0.859\text{--}0.892)$$

Give it one infected and one uninfected patient: it picks the infected one 88% of the time.

By recorded sex: 0.878 in women, 0.875 in men. Paired difference $+0.004$ (-0.030 to $+0.038$), $p = 0.82$.

Every discrimination check passes. Every parity check on sex passes.

And the model is not ready to deploy.

The model says 20%. The real risk is about 8%.

Take the model's output at face value, and compare it with what actually happened to those patients:

The model shows	5%	10%	20%	30%
The real risk is	1.9%	3.9%	8.3%	13.4%

A factor of two to three, everywhere on the scale. Averaged over the cohort the risk shown sits **6 percentage points** from the risk observed, in a population where 4.7% of patients get infected — which buys antibiotics, monitoring and delayed discharge for people whose real risk is a third of what the screen says.

The previous slide was not wrong. It measured something else.

The model puts patients in very nearly the right *order*. It is the *values* it prints that are wrong. Only the first of those was checked.

And it is not the same in every patient group

Split by physiological reserve — roughly, how much complication a patient can absorb — and the same 20% on the screen means four different things:

Stratum	Patients	Infections	Shown 20%, really...
Overall	10,719	504	8.3%
Frail	6,139	400	9.6%
Pre-frail	3,558	98	6.1%
Non-frail	1,022	6	2.1%, and unsupported

Six infections is too few to put an interval around anything, so that last row is a number we show and do not interpret (§7). The frail–pre-frail contrast is estimable, and it is real.

The name for all of this is **calibration**; §5 makes it precise. What matters here is the consequence: in one stratum, acting on this score at the intended threshold does *more harm than not using it at all* (§6). No amount of discrimination could have told you that.

What this deck is

A recipe you can run on *your* model, in the order the questions actually arise:

1. What you must bring to the audit, and what disqualifies your inputs.
2. Three axes: what each measures, the estimator, and how to read it.
3. Uncertainty: what the interval is conditional on — the part that is easiest to get quietly wrong.
4. Mitigation, in the order you should try it.
5. A report you can hand to a reviewer.

Most slides end with a **Trap** — the mistake that slide exists to prevent — and the ones with a formula also carry an **In words** line, which says the same thing without the notation.

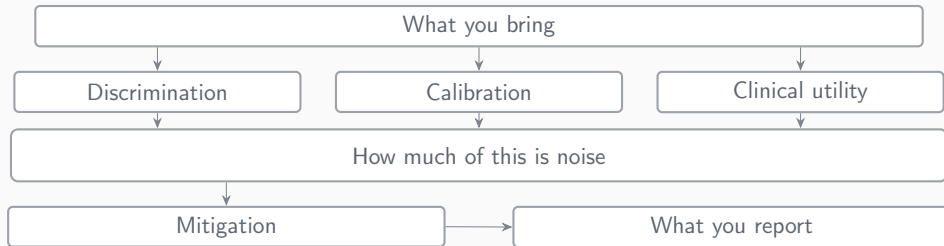
Code marked **v0.4** needs isitfair 0.4; unmarked code runs on 0.3.0.

Six words you need before we start

Not fairness vocabulary — just prediction-model evaluation. Everything after this is built out of these.

- **Rank vs. number.** A score can be right about the *order* of patients and wrong about every *value* it prints. The spine of the deck.
- **Threshold t .** What turns a score into a decision. Anything defined at a threshold describes the decision rule, not the model.
- **Prevalence.** How common the event is in a group. It moves several metrics on its own, model unchanged — so those metrics do not compare across groups.
- **Log-odds.** $\log \frac{p}{1-p}$ stretches $[0, 1]$ onto the whole real line, where “every risk is too high” is one number instead of a curve.
- **Bootstrap interval.** Resample your patients, recompute a few thousand times, keep the middle 95%: how far would this move on a different sample?
- **Not estimable.** Too few events to support an interval. The point estimate is still shown; the interval is withheld.

The three axes



Three different questions

Discrimination

Do sick patients
get higher scores?

AUROC, AUPRC
TPR, FPR, PPV, NPV

Calibration

Is a 10% score re-
ally a 10% risk?

curve, α , β
Brier, ECE, ICI, H-L

Clinical utility

Does acting on it do
more good than harm?

net benefit, sNB
decision curve

They are not redundant and they do not imply one another. A model can rank perfectly and be badly calibrated; it can be well calibrated and useless at every threshold a clinician would accept.

Trap. Reporting only the first because it is the one everyone recognises.

Within-group *and* gap — never gaps alone

For each axis you get two kinds of number:

- **Within-group:** this subgroup's own AUROC, its own α , its own net benefit.
- **Gap:** the spread across subgroups on the same axis.

A gap of zero is not good news by itself

Two subgroups can have identical sensitivity because the model is equally useless in both. Equal harm is parity. Reporting only gaps makes that outcome look like success.

This is why every `isitfair` frame returns subgroup rows *and* a GAP row, and why the API never offers a gaps-only mode.

Trap. A fairness dashboard that shows four gap bars and no absolute performance.

Which axis is *primary* depends on your use case

Decide this before you look at any output, and write it down.

Score used as a probability

(risk shown to a clinician, threshold-based triage)

- Primary: **calibration** and **clinical utility**
- Threshold-specific parity gaps are *descriptive*

Score used as a ranking

(fixed-capacity queue, top- k review)

- Primary: **discrimination** and error-rate parity
- Calibration matters less; ordering is the product

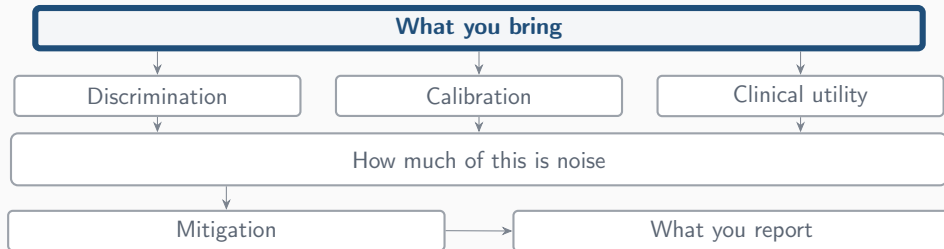
A raw subgroup difference is not evidence of model failure. Even an optimal predictor shows unequal subgroup performance when covariate or outcome distributions differ. Interpret differences across clinically distinct strata *in the context of case mix and prevalence* — consistently, for every axis, not just the ones with a convenient explanation.

What counts as a subgroup axis

- **Prespecified from clinical knowledge and the deployment context.** Not discovered from the data. Subgroup discovery is a different research programme with different error control.
- **Available at the moment of prediction.** An axis you only learn afterwards cannot inform the decision you are auditing.
- Mix **sociodemographic** axes (recorded sex, age stratum) with **clinical** ones (physiological reserve, comorbidity burden). The clinical axis is often where the failure actually is.
- Keep **case-mix reference axes** (specialty, procedure cluster) separate in your reporting. They characterise the population; they are not patient subgroups you owe fairness to.

Trap. Auditing only the protected attributes. In the worked example, sex was clean on every axis and physiological reserve was where the model broke.

Setting up your audit



The four things you must bring

1. `y_true` — binary outcome, $Y_i \in \{0, 1\}$, one per observation.
2. `y_proba` — your model's predicted probability, $p_i \in [0, 1]$. **Out-of-fold or out-of-sample.** See next slide.
3. `sensitive_features` — a dict of label arrays, *separate from your feature matrix*. The package never sees X .
4. `threshold` — one prespecified operating threshold $t \in (0, 1)$.

Notation used throughout

For subgroup g : I_g its observations, $n_g = |I_g|$, n_g^+ its events, $\pi_g^+ = n_g^+/n_g$ its prevalence. At threshold t an observation is *flagged* when $p_i \geq t$, giving $TP_g(t)$, $FP_g(t)$, $FN_g(t)$, $TN_g(t)$.

Why the predictions must be out-of-fold

If a model saw an observation during training, its prediction on that observation is optimistic — and *differentially* optimistic, because overfitting is worst where data are sparse, which is exactly your small subgroups.

- Cross-validated? Use the held-out prediction from the fold that excluded each observation.
- Any resampling (SMOTE, class weights) belongs **inside the training fold, after fold assignment**, never applied to held-out rows.
- Recalibrating? That transform is also fitted, so it needs the same treatment — see the cross-fitting slide.

Trap. Class rebalancing shifts calibration hard. In the worked example, removing SMOTE moved calibration-in-the-large (§5) by $+0.94$ logits — but did *not* shrink the between-stratum gradient. Rebalancing explains the *offset*, not the *gap*. Do not assume it explains both.

Constructing the audit

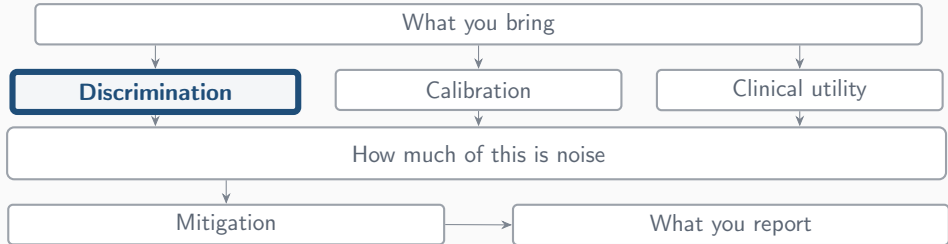
```
import numpy as np
from isitfair import FairnessAudit

audit = FairnessAudit(
    y_true=y,                # 0/1, shape (n,)
    y_proba=p_oof,          # out-of-fold probabilities, shape (n,)
    sensitive_features={
        "sex": sex,          # marginal axes: string keys
        "age_group": age_group,
        "reserve": reserve,
        ("sex", "reserve"): None, # intersection; None = build labels for me
    },
    threshold=0.10,         # prespecified operating threshold
    n_bootstrap=2000,
    random_state=42,        # same seed + same data = identical output
)
```

Nothing is computed here. Work happens when you call an axis, and results are cached.

Trap. A tuple key whose constituents are not *also* present as string keys raises. Intersections are defined relative to their marginals.

Discrimination



AUROC — can the model tell them apart?

$$\text{AUROC}_g = \Pr(p_i > p_j \mid i, j \in I_g, Y_i = 1, Y_j = 0)$$

In words. Pick one patient who got infected and one who did not, both from *this* subgroup. AUROC is how often the model gives the infected one the higher score. Estimated by the Mann–Whitney rank-sum statistic on I_g , rescaled to $[0, 1]$, ties averaged.

- Threshold-free: it summarises the whole ranking.
- Prevalence-insensitive: it is a within-class comparison, so it barely moves when the event rate changes. That is a feature here and a liability in the next slide.

Trap. Comparing subgroup AUROCs as if they were on a common scale. Each is computed *within* its own subgroup, so it never tells you whether the two groups' scores are mutually comparable.

AUPRC — the low-prevalence complement

The area under the within-subgroup precision–recall curve: a trapezoidal sum over the operating points $(\text{TPR}_g(t), \text{PPV}_g(t))$ swept across all observed thresholds.

In words. Of the patients you would flag, how many really have it — traced across every threshold at once. AUROC asks about ordering; this asks what the flagging costs.

- The no-skill baseline for AUROC is always 0.5. For AUPRC it is π_g^+ — the subgroup's own prevalence.
- So $\text{AUPRC} = 0.38$ is excellent at 4.7% prevalence and mediocre at 30%. **Always read it against π_g^+ .**
- At low prevalence AUROC is dominated by the easy true negatives; AUPRC is not, which is why the two can disagree about which subgroup is better served.

Trap. Ranking subgroups by AUPRC. You are ranking their prevalences.

The rates at your threshold

Once you fix t , the model becomes a decision rule and four rates follow:

$$\begin{aligned} \text{TPR}_g &= \frac{TP_g}{TP_g + FN_g}, & \text{FPR}_g &= \frac{FP_g}{FP_g + TN_g}, \\ \text{PPV}_g &= \frac{TP_g}{TP_g + FP_g}, & \text{NPV}_g &= \frac{TN_g}{TN_g + FN_g}, & \text{TNR}_g &= 1 - \text{FPR}_g. \end{aligned}$$

In words. TPR and FPR read across **the truth**: of those who did get infected, what share did we catch; of those who did not, what share did we flag anyway. PPV and NPV read across **the prediction**: of those we flagged, what share really had it, and the mirror image. The first pair are properties of the model; the second pair move with prevalence even when the model is unchanged. **Trap.** Comparing PPV across subgroups with different prevalence and calling the difference bias. Two strata at 1% and 10% prevalence *must* differ in PPV at a common threshold. That is arithmetic, not discrimination.

Running it, and what comes back

```
disc = audit.discrimination()
disc[disc.subgroup != "GAP"][
    ["attribute", "subgroup", "n", "n_events", "prevalence",
     "auroc", "auroc_ci_low", "auroc_ci_high"]
]
```

One long-format DataFrame. Per attribute you get one row per subgroup, one Overall row, and one GAP row.

Column group	Contents
keys	attribute, subgroup, shrunk
size	n, n_events, prevalence
metrics	auroc, auprc, sensitivity, specificity, ppv, npv each with _ci_low / _ci_high
gap	gap_metric, gap_value, gap_ci_low, gap_ci_high v0.4
reporting	n_replicates, n_estimable, proportion_nonestimable, event_floor v0.4 , below_floor v0.4 , not_estimable

The four parity gaps

For a pair of subgroups g, g' , all evaluated at the same t :

$\Delta_{\text{EOpp}} = \text{TPR}_g - \text{TPR}_{g'}$	equal opportunity (sensitivity gap)
$\Delta_{\text{PE}} = \text{FPR}_g - \text{FPR}_{g'}$	predictive equality (false-alarm gap)
$\Delta_{\text{SP}} = \frac{\text{TP}_g + \text{FP}_g}{n_g} - \frac{\text{TP}_{g'} + \text{FP}_{g'}}{n_{g'}}$	statistical / demographic parity (flag rate)
$\Delta_{\text{EO}} = \max(\Delta_{\text{EOpp}} , \Delta_{\text{PE}})$	equalized odds

In words. One rate, subtracted between two groups, at one threshold: caught cases, false alarms, and how often you flag at all. Over more than two levels, the largest absolute value across pairs.

In the frame: `equal_opportunity_difference`, `predictive_equality`, `statistical_parity`, `equalized_odds_gap`.

Trap. All four are properties of *the decision rule at one threshold*, not of the model. Change t and they change.

Cohen's h — is the gap big, or just measurable?

$$h(p_g, p_{g'}) = 2(\arcsin\sqrt{p_g} - \arcsin\sqrt{p_{g'}})$$

In words. A difference of two proportions, rescaled so the same arithmetic gap counts for more near 0 or 1 than in the middle.

- A 2-point sensitivity gap means something different at 90% than at 10%. h knows that (the arcsine transform stabilises a proportion's variance); Δ does not.
- Conventional bands: 0.2 small, 0.5 medium, 0.8 large.

Where no effect size applies

Composite gaps (equalized odds) and non-rate metrics (calibration intercept and slope, net benefit) have no single matching effect size. Leave the column **blank** rather than filling it with a misleading number.

Trap. A large- n cohort makes almost any gap significant.

Reading a gap: three honest verdicts

1 — Descriptive heterogeneity

The strata are clinically different populations. Procedure clusters differ in infection risk; age strata differ in physiology. A gap here is expected and is context, not a defect. *Apply this reading consistently* — you may not invoke case mix only for the gaps you find inconvenient.

2 — A deployment-relevant limitation

The score *as used* misleads in one stratum: miscalibrated probabilities, or negative net benefit at the intended threshold. This is the finding that changes what you do next.

3 — Not estimable

Too few events to support any claim. Report the point estimate, withhold the interval, say so. Covered in the uncertainty section.

The trap this axis exists to set

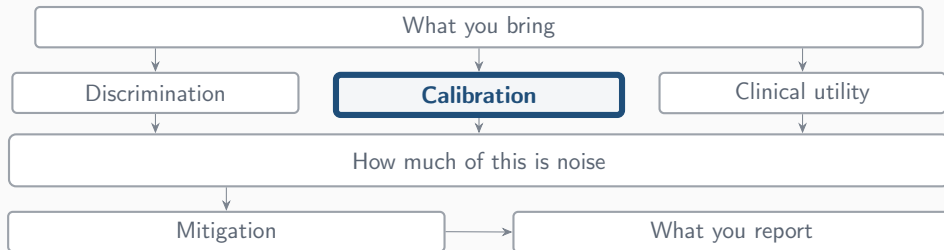
From the worked example, at $t = 0.10$:

	n	AUROC	Sens.	Spec.
Age 0–17	367	0.807	0.857	0.462
Age 18–44	941	0.854	0.824	0.682
Age 45–64	3,715	0.888	0.918	0.666
Age 65–79	4,684	0.889	0.912	0.654
Age ≥ 80	1,012	0.805	0.849	0.577

Sex was flawless: $\Delta\text{EO} = 0.010$. Age gives $\Delta\text{EO} = 0.220$ — the largest gap over *all* pairs on the axis, produced by the two marked rows, on **specificity**. One global threshold flags far more non-infected children.

Trap. Auditing only the axis you were asked about. Then declaring the model fair.

Calibration



What calibration means

A model is calibrated within subgroup g if, among observations assigned predicted risk p , the observed event rate is also p :

$$\mathbb{E}[Y \mid p, A = g] = p$$

In words. Among everyone the model called 10%, about one in ten should have had it. This is the axis that matters when a **number** reaches a clinician or a threshold; ranking cannot substitute for it.

- **Overestimation** → unnecessary monitoring, delayed discharge, avoidable antibiotics.
- **Underestimation** → patients who needed surveillance and did not get it.

Assess it three ways — visually, parametrically, with aggregate scores; each misses what the others catch.

1 — The calibration curve

Estimate $\mathbb{E}[Y \mid p, A = g]$ as a smooth function of p by LOESS, with 95% bootstrap bands.

In words. At each point on the risk scale, ask what share of the *nearby* patients had the event. LOESS does that with a sliding window; its span is a tuning choice, exactly as ECE's bin count is.

- The identity line $y = p$ is perfect calibration; **below** it the model overestimates risk, **above** it underestimates.
- Clip each curve to *its own* stratum's observed predicted range. Drawing beyond it renders extrapolation as a fitted result.

It is the only calibration output that shows *where* the model fails: no single number separates “wrong everywhere by a little” from “fine except at the top decile”.

Trap. Reading a wiggle in a sparse stratum as structure. Check the band, and check the event count.

2 — Calibration-in-the-large: the intercept α

On the log-odds scale, $l_i = \log \frac{p_i}{1-p_i}$, fit one parameter with the slope **fixed at 1**:

$$\text{logit Pr}(Y_i = 1) = \alpha_g + l_i, \quad i \in I_g$$

In words. One number for “every predicted risk is off by the same amount”. $\alpha_g = 0$ is unbiased on average; $\alpha_g < 0$ is systematic **over**prediction, $\alpha_g > 0$ systematic underprediction.

Fixing the slope is the point: it asks whether the whole risk scale is shifted, without letting a slope change absorb the shift.

Trap. α is **prevalence-dependent**. If your cohort’s event ascertainment changes across time or across strata, α moves for reasons that have nothing to do with the model. Report the window you computed it in.

3 — The slope β

Now free both terms:

$$\text{logit Pr}(Y_i = 1) = \alpha_g^{(2)} + \beta_g l_i, \quad i \in I_g$$

In words. Are the risks spread too wide, or squeezed too narrow? Not where they sit on average — that is α 's job.

- $\beta_g = 1$: the risk gradient is correctly scaled.
- $\beta_g < 1$: **too spread out** — the overfitting signature.
- $\beta_g > 1$: **too compressed** — what class rebalancing leaves behind, and what this model shows.

$$\alpha_g^{(2)} \neq \alpha_g$$

The free fit's intercept is not calibration-in-the-large. Never quote it as α .

Reading α and β together

	$\beta < 1$ (<i>too spread out</i>)	$\beta \approx 1$	$\beta > 1$ (<i>too compressed</i>)
$\alpha \approx 0$	Risks too extreme at both ends; average right. Needs a slope correction.	Calibrated. Nothing to fix.	Risk range squeezed toward the middle; average right. Slope correction.
$\alpha < 0$	Both problems. Fix the shift first.	Pure shift. Every risk too high by a constant on the logit scale. A common recalibration fixes this.	Both problems , and where the worked example sits. Fix the shift first, then look again — the residual is what actually needs group-specific work.

In the worked example: $\alpha = -1.02$ and $\beta = 1.57$ — a large shift on top of a badly compressed risk range, the bottom-right cell. Common recalibration was still the right first move, and it fixed both: it took α to -0.007 and β to 0.93 without using group membership at all.

Trap. Reaching for group-specific correction before you have removed the shift that affects everyone.

4 — Brier, ECE, ICI: three aggregate scores

Brier score — accuracy and calibration together

$$\text{BS}_g = \frac{1}{n_g} \sum_{i \in I_g} (p_i - Y_i)^2$$

The average squared miss. A *proper* scoring rule — it is optimised only by reporting your true belief — but **prevalence-dependent**: a rare-event stratum earns a low Brier almost for free, so it does not compare across strata.

Expected calibration error — binned

$$\text{ECE}_g = \frac{1}{n_g} \sum_{k=1}^K n_{g,k} |\bar{p}_{g,k} - \bar{y}_{g,k}|$$

Bin patients by predicted risk ($K = 10$, equal frequency) and average how far each bin's predicted rate sits from its observed rate. Depends on your binning.

ICI — the binless version, and how to use all three

Integrated calibration index

$$\text{ICI}_g = \frac{1}{n_g} \sum_{i \in I_g} |f_g(p_i) - p_i|$$

The same average gap, measured per patient against the smooth calibration curve f_g rather than inside bins. Nothing to argue about but the smoother.

Reads as. ECE and ICI usually agree closely. When they *disagree*, your binning is doing the work — look at the curve before you report either.

Trap. Using Brier to compare strata. It rewards low prevalence. Use it within a stratum, before and after mitigation, where prevalence is held fixed.

5 — Hosmer–Lemeshow: what the test actually asks

Sort every patient by predicted risk, cut the list into $g = 10$ groups of roughly equal size, and in each group compare the events the model **expected** (its predicted risks, added up) with the events there actually **were**.

Null hypothesis: in every group, expected = observed

The statistic adds up the discrepancies, and the p -value says how surprising that total would be if the model really were calibrated.

A large p is not evidence of calibration

Small p : evidence of miscalibration. Large p : *no evidence either way*. A stratum with 30 events will pass this test almost whatever the model does to it, because it has no power to fail. Never report “H–L was not significant, so the model is calibrated in this subgroup.”

Trap. Treating the test as pass/fail in either direction.

... and the degrees-of-freedom question

With O and E the observed and expected events per group:

$$\chi_{\text{HL}}^2 = \sum_{k=1}^g \frac{(O_k - E_k)^2}{E_k \left(1 - \frac{E_k}{n_k}\right)}$$

In words. Each group's miss, squared, scaled by how much it could have missed by chance, referred to χ_{df}^2 . Which df is not settled.

Use $df = g$, not $g - 2$

The $g - 2$ convention spends two degrees of freedom recovering an intercept and slope *when the model was fitted on the data under test*. Out-of-fold predictions estimate nothing, so nothing is spent — and $g - 2$ here roughly doubles the rate of miscalibration you declare that is not there.

Trap. H-L grows with n at fixed lack of fit, so it does **not compare across strata of different size**. Read it beside ECE, ICI and the per-group table.

Running calibration

```
cal = audit.calibration()
# calibration_intercept, calibration_slope, brier (each +_ci_low/_ci_high),
# ece, ici, gap_metric, gap_value, gap_ci_low, gap_ci_high

hl = audit.hosmer_lemeshow()                # + ece, ici columns    [v0.4]
# hl_statistic, hl_df, hl_p_value, n_bins_used, bins_reduced,
# decile_slope, decile_intercept

deciles = audit.hosmer_lemeshow(detail=True)
# per risk group: observed, expected, observed_rate, expected_rate, difference

curves = audit.calibration_curves()          # LOESS + bootstrap band
```

`df_mode` defaults to “validation” ($df = g$). Pass `df_mode=“development”` only if you really did fit on this data.

Trap. Publishing an H–L statistic without the per-group table. The statistic says *that* it misfits; only the table says *where*.

Calibration gaps

Two gap statistics, both max-minus-min across the subgroups of an axis, both with bootstrap intervals:

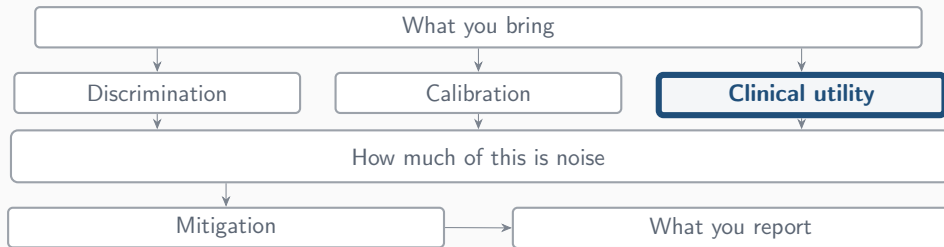
- `max_intercept_gap` — the spread in systematic bias.
- `max_slope_gap` — the spread in how the risk scale is stretched.

The gap is filed under its own name

A gap has no event count of its own. It is recorded as `max_intercept_gap`, not as a calibration-intercept row, and it **inherits the reportability failure of either contributing stratum**. In the worked example the largest reserve gap (1.62 logits) involves a six-event stratum, so it carries no interval — while the estimable frail-pre-frail contrast, 0.49 (0.28 to 0.71), does.

Trap. Quoting the biggest gap. The biggest gap is usually the one resting on your smallest stratum.

Clinical utility



The question a decision curve answers

Not “is the model accurate?” but:

If I act on this score at threshold t , am I better off than if I acted on nobody — or on everybody?

You need three curves on one plot:

- **The model** at each threshold.
- **Treat none** — the zero line.
- **Treat all** — act on everyone regardless of score.

Treat-all must use each subgroup's own prevalence

A treat-all reference computed from the whole cohort is the wrong benchmark for every subgroup whose prevalence differs from it — which is all of them. This is a real defect that got caught in peer review.

$$\text{NB}_g(t) = \frac{TP_g(t)}{n_g} - \frac{FP_g(t)}{n_g} \cdot \frac{t}{1-t}$$

In words. Count the patients you correctly flagged, then subtract the false alarms — each one discounted by what your own threshold says a false alarm costs relative to a miss.

- Units: net true positives per patient. Multiply by 1000 to report “per 1,000 patients”, which is how clinicians read it.
- $\text{NB}_g(t) > 0$: better than treating nobody.
- $\text{NB}_g(t) > \text{NB}_g^{\text{treat-all}}(t)$: better than treating everybody.
- $\text{NB}_g(t) < 0$: **acting on the score is worse than ignoring it** in that subgroup.

This is the axis that turns a calibration finding into a deployment decision.

Where $t/(1 - t)$ comes from — and what it is not

Choosing t is choosing an exchange rate. If you would act at $t = 0.10$, you are saying one missed event is worth

$$\frac{t}{1 - t} = \frac{0.10}{0.90} = \frac{1}{9}$$

... i.e. a missed event costs about nine unnecessary interventions.

This is arithmetic, not a preference you elicited

The 1:9 weighting *follows mathematically* from picking 0.10. It is not evidence that clinicians hold that trade-off. Say which one you have.

So treat your threshold as a **reference operating anchor**, not an evidence-derived optimum, and back it up:

- Sweep a plausible band (e.g. 0.05–0.20) and tabulate net benefit at each.
- Draw the curve wider still (0.02–0.40) so a reader sees the shape.

Standardized net benefit

Net benefit scales with prevalence, so raw NB is not comparable across strata. Divide by the subgroup's own prevalence:

$$\text{sNB}_g(t) = \frac{\text{NB}_g(t)}{\pi_g^+}$$

In words. What fraction of the benefit a perfect model could deliver in this group are we actually capturing? $\text{sNB} = 1$ is as good as a perfect model on I_g ; $\text{sNB} \leq 0$ is no better than treating nobody. **Reads as.** $\text{sNB} = 0.45$ in frail and -0.10 in pre-frail means the model captures nearly half the available benefit in one group and destroys value in the other — at the *same* threshold.

Trap. Comparing raw NB across strata. The low-prevalence one has less to work with.

The utility gap, and the zero crossing

Primary clinical-utility gap

$$\Delta\text{NB}_{\max} = \max_{t \in [0.02, 0.40]} \max_{g, g' \in G} |\text{NB}_g(t) - \text{NB}_{g'}(t)|$$

The worst disagreement between any two groups, anywhere in the threshold band you would plausibly deploy at — with a percentile interval from the same bootstrap replicates.

Zero crossing v0.4

The threshold at which a subgroup's net benefit crosses zero — i.e. where the model stops helping that group. Solved in closed form per constant-flagged-set segment, $t^* = TP/(TP + FP)$, with jump discontinuities reported separately.

Reads as. “No crossing in range” is the good answer. A crossing at $t = 0.195$ means the model turns harmful for that stratum inside the band you would plausibly deploy at.

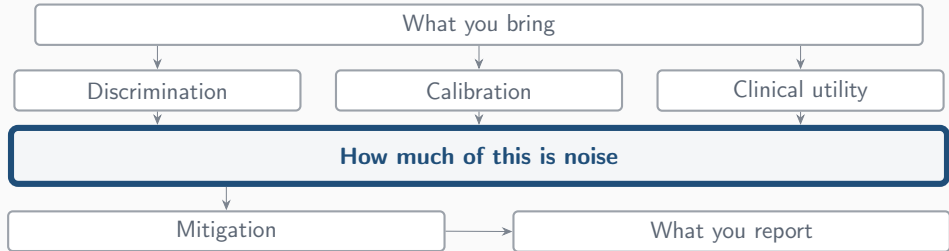
Running clinical utility

```
dca = audit.decision_curve(  
    thresholds=[0.05, 0.075, 0.10, 0.125, 0.15, 0.175, 0.20],  
    zero_crossing=True,                                # [v0.4]  
)  
# threshold, net_benefit (+_ci_low/_ci_high),  
# standardized_net_benefit (+_ci_low/_ci_high [v0.4]),  
# treat_all_nb, treat_none_nb, zero_crossing [v0.4],  
# gap_metric, gap_value, gap_ci_low, gap_ci_high  
  
curve = audit.decision_curve()                        # default grid: 0.02 -> 0.40, 20 points
```

`treat_all_nb` is already per-subgroup. Report NB per 1,000 by multiplying; the frame stores it per patient.

Trap. Reporting utility at one threshold only. If the sign of your finding flips at 0.075, a single-threshold table is a claim you cannot defend.

Uncertainty, floors and multiplicity



What the bootstrap resamples — and why it matters

Every interval is a 95% percentile interval over $B = 2000$ resamples. The question is **what varies between replicates**.

“patient” (*default*)

Resample patients with replacement *within each subgroup stratum*. Subgroup n is preserved; its **event count varies**.

Intervals reflect sampling variability in the observed prevalence — which is real variability.

Defensible for rank-based metrics. **Understates uncertainty for every prevalence-dependent metric** — calibration intercept and slope, Brier, PPV/NPV, net benefit, sNB.

“case_control” (*frozen*)

Resample cases and non-cases separately at fixed sizes. Every replicate carries the **observed event count**.

Estimates uncertainty *conditional on the observed prevalence*.

The defect this caused, concretely

A stratum with **6 events**. Calibration intercept, two schemes:

Scheme	95% CI	Width (logits)
case_control	(−2.55, −2.40)	0.15
patient	—	1.76

More than tenfold wider. And the narrow one is not conservative — it is answering a question nobody asked: “how uncertain is this, *given* that exactly six events occur?”

Structural consequence

case_control **cannot produce a zero-event replicate**. A subgroup with one event has exactly one in every replicate. So your non-estimability diagnostics read 0% by construction — they cannot warn you.

Use ‘patient’. The frozen path exists only to reproduce published analyses.

Three properties your bootstrap should have

1 — One overall interval, reused everywhere

The whole-cohort row is resampled *once* and reused across every subgroup axis. If your Table 2 shows a different overall CI in the sex section than in the age section, you re-ran the bootstrap per axis — and a reviewer will notice.

2 — Gaps are reductions, not a second draw

All subgroups of an axis share *one* replicate index array, so a gap distribution is computed from the stored per-subgroup replicates. That preserves the dependence between a gap and the rates it is built from. Resampling again would break it.

3 — Addressable, order-independent randomness

Replicate *b* of attribute *a* is seeded from `(entropy, stream, attribute_id, b)`. Results do not depend on dict order, chunk size, or worker count. `random_state=42` on identical data gives bit-identical output.

Rule 1 — the replicate rule

Some replicates simply cannot produce a value: a resample containing no events has no AUROC and no calibration intercept.

- Those replicates are recorded as NaN, **never silently dropped**.
- If **more than 10%** of replicates are non-estimable, the cell is reported as *not estimable* rather than given an interval.
- `n_replicates`, `n_estimable`, `proportion_nonestimable` are on every frame, so the accounting is auditable.

Read the fine print on this one

Percentile intervals are computed over *estimable replicates only*. So an interval is conditional on estimability — and therefore narrower than the true marginal interval — from the *first* non-estimable replicate, not from 10%. The 10% figure is a reporting cutoff, not a validity guarantee.

Rule 2 — the event floors

A metric needs a minimum number of **events**, not observations:

Metric	Minimum events
AUROC	25
Calibration intercept	25
Calibration slope	100
Net benefit	50

In words. Below this many events the metric is mostly noise, so you fix the cutoff before you look. The slope floor is highest because it estimates a *gradient*, not just a level; AUROC's standard error is roughly $0.5/\sqrt{E}$ here, which is where 25 comes from.

Trap. These are *your* pre-registration. Adopt them or set your own with a written rationale — but decide before the analysis, because afterwards every incentive points at an exception.

The conservative union — and what suppression means

not reportable = (below its event floor) **OR** ($> 10\%$ replicates non-estimable)

- A cell below its floor is suppressed **even if every replicate estimated cleanly**. The floor is not a diagnostic; it is a rule.
- A gap inherits the floor failure of **either** contributing stratum.
- Apply it without exception — *including where it suppresses your own headline finding*. That is the whole value of prespecifying it.

Suppression withholds the interval, never the point estimate

A point estimate is a real quantity that was computed. Deleting it hides what you audited. Show it, label it *not estimable*, do not interpret it inferentially. In figures: draw it dashed with a hollow marker — present, and visibly unsupported.

Estimability in code

```
audit = FairnessAudit(
    ..., resample_scheme="patient", n_bootstrap=2000, random_state=42,
    nonestimable_threshold=0.10,          # rule 1
    event_floors={                        # rule 2                                [v0.4]
        "auroc": 25, "calibration_intercept": 25,
        "calibration_slope": 100, "net_benefit": 50,
    },
)

est = audit.estimability()                # [v0.4]
# attribute, subgroup, metric, n, n_events, event_floor, below_floor,
# n_replicates, n_estimable, proportion_nonestimable, not_estimable_final

est[est.not_estimable_final]             # everything you may not interpret
```

Persist this table beside your results and have downstream code *read* it rather than re-deriving a floor. A rule restated in five places becomes five different rules.

Multiplicity: correct within families, not globally

Control the false discovery rate at $q = 0.05$ with Benjamini–Hochberg, *separately within each prespecified family*: order $p_{(1)} \leq \dots \leq p_{(m)}$, find the largest k^* with $p_{(k)} \leq (k/m) q$, reject everything up to it.

In words. The more tests in a family, the higher the bar each has to clear. FDR budgets the *share of your findings* allowed to be false — looser than guarding against *any* false one, and the right trade for a screen.

Typical families: gap tests within each axis; an interaction screen (e.g. sex $\times$ comorbidity, 23 tests); the Table 1 scan; a feature-by-attribute effect-size scan.

Within-subgroup point estimates are reported without correction. They are descriptions, not hypothesis tests.

Trap. Pooling every p -value in the paper into one family. You lose the findings that mattered to protect against tests you never intended to make.

Interaction tests: is the *effect* different, or just the rate?

A gap describes what a fixed threshold sees. It cannot tell you whether a covariate's *effect* differs by attribute. For that, compare nested logistic models and test $H_0: \beta_{XA} = 0$:

$$M_0 : \text{logit Pr}(Y=1) = \beta_0 + \beta_X X + \beta_A A + \beta_C^\top C$$

$$M_1 : \text{logit Pr}(Y=1) = \beta_0 + \beta_X X + \beta_A A + \beta_{XA}(X \cdot A) + \beta_C^\top C$$

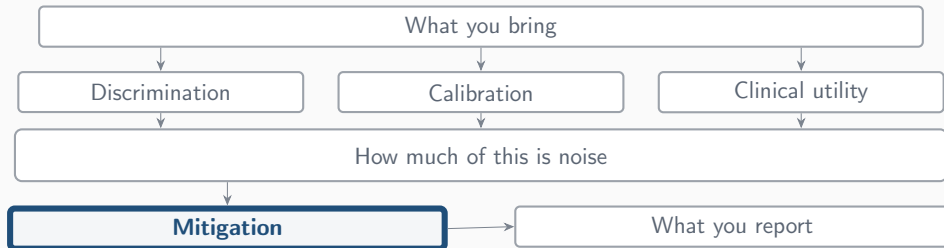
$$\text{LR} = 2(\ell(M_1) - \ell(M_0)) \sim \chi_{df}^2$$

```
from isitfair import interaction_test, fdr_correct, odds_ratio_2x2
res = interaction_test(df, outcome="y", sensitive="sex",
                      covariate="comorbidity",
                      adjust_for=("age_years", "asa_class")) # NUMERIC columns
res.p_value, res.delta_r2      # always report both
```

In words. Does the extra term buy enough fit to be worth a parameter? `delta_r2` is how much of Nagelkerke's R^2 it added — report it, because at this n a tiny effect is still significant. `adjust_for` columns enter as **linear terms** and must be numeric.

Trap. A within-group odds ratio is an *association*, not evidence of a difference between groups. Only the interaction term is.

Mitigation



Climb the ladder — do not jump to the top

Rung 0 — none. The baseline. You cannot claim an improvement without it.

Rung 1 — common, cohort-wide. One calibrator for everyone. Fixes the shift that affects all patients.

Rung 2 — group-specific. One calibrator per subgroup. Only justified by what *survives* rung 1.

In the worked example: common recalibration moved α from -1.020 to -0.007 and left a frail-pre-frail gap of 0.308; group-specific reduced that residual to 0.005.

Trap. Skipping rung 1. Group-specific correction applied to a cohort-wide shift will “work” — and you will have attributed a global problem to a subgroup, and built a model that uses group membership for no reason.

Isotonic recalibration

Per subgroup g , fit a monotone non-decreasing step function on the calibration set:

$$\iota_g = \arg \min_{\iota \text{ monotone}} \sum_{i \in I_g^{\text{cal}}} (\iota(p_i) - Y_i)^2, \quad p_i^{\text{iso}} = \iota_g(p_i)$$

In words. Learn a staircase that only ever goes up, so the order is untouched and only the values move.

- **Ranking and AUROC are preserved** — bar tie-induced rank collapses at very low event counts.
- Non-parametric: it fixes shapes a logistic recalibration cannot, and its output is still a probability, so everything downstream keeps working.

Trap. It **overfits sparse strata**. Inspect the post-recalibration slope and the event count together.

Cross-fitting: the transform is a fitted model too

A recalibrator estimated on the same rows you then evaluate is in-sample. It will look excellent and mean nothing.

```
ladder = audit.recalibration_ladder(                                     # [v0.4]
    method="isotonic",
    by="reserve",
    folds=published_fold_ids,    # reuse the model's OWN CV folds
    random_state=42,
)
# rung ("none" / "common" / "group_specific") x stratum, with
# calibration_intercept, calibration_slope, ece, ici, brier,
# net_benefit at t, and the between-stratum intercept gap
```

Each transform is fitted on 4 folds and applied to the held-out fold, so every observation gets an out-of-fold recalibrated prediction and the before/after comparison is on the full cohort.

Trap. Cross-fitting on *fresh* folds when the model itself was cross-validated. Pass the model's own fold ids — otherwise the calibrator has seen rows the model was tested on, and your leakage profile is different from the one you documented.

When the ladder tells you to stop

The worked example's third rung was **worse than useless** in one stratum:

- Non-frail: **6 events**, below every prespecified floor.
- Its fitted isotonic map produced only a handful of distinct probabilities.
- Post-recalibration calibration slope: **0.12** (against 0.93 overall).

That is a degenerate transform. Its average calibration looks corrected because the map has collapsed to nearly a constant.

The conclusion that follows

The residual gradient after common recalibration was similar in magnitude to the upper bound attributable to differential outcome ascertainment across strata. So: adopt **common** recalibration, and **defer** group-specific correction until the endpoint is measured comparably in every stratum. Do not encode a data-quality artefact as a group-specific model.

Score-distribution parity, and why it is a comparator

The 1-D Wasserstein barycenter maps every subgroup's score distribution onto a common scale:

$$Q_{\text{bary}}(u) = \sum_{g \in G} w_g F_g^{-1}(u), \quad p^W = Q_{\text{bary}}(F_g(p))$$

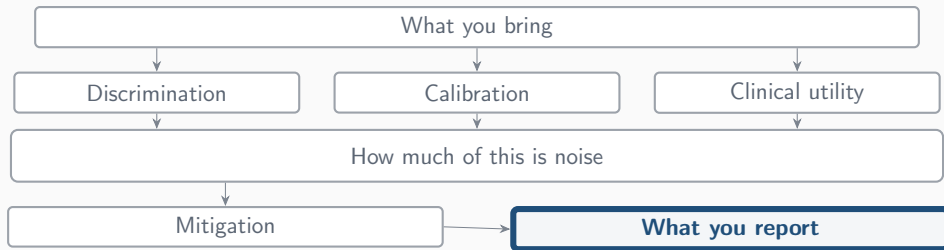
In words. Line every subgroup's scores up by percentile and average them onto one common scale — monotone within each subgroup, so within-group AUROC survives exactly.

```
from isitfair import WassersteinPostprocessing
w = audit.mitigate(method=WassersteinPostprocessing(
    constraint="demographic_parity"), calibration_data=calib)
```

It equalises distributions, not risks — so it breaks calibration

Calibration and equalized odds cannot both hold when group base rates differ — not as a matter of estimation but as a theorem: no non-trivial score satisfies both (Pleiss et al., 2017). Constrain this to demographic parity, treat it as a **parity comparator**, and expect calibration to get worse. If your failure mode is miscalibration, this is not your fix.

Reporting, and the whole thing at once



The report

```
audit.report(  
    out="audit_report.html",  
    title="My Model - Subgroup Deployment-Readiness Audit",  
    metadata={"Model": "XGBoost v3", "Cohort": "n=8,412 (2019-2024)",  
              "Threshold": "0.10", "Predictions": "out-of-fold, 5-fold CV"},  
    mitigation_audits=ladder_audits,      # optional before/after section  
    table_one_data=cohort_df,            # optional cohort characterisation  
)
```

Self-contained HTML: plots embedded, no JavaScript, print-friendly. It also drops 300-DPI PNGs of every figure into `audit_report_figures/` for your manuscript.

With `random_state` fixed, the report is **bit-identical** across runs. That is what makes “regenerate the report from the serialized audit” a defensible reproducibility claim rather than a hope.

What is in it

1. Header + your metadata
2. Audit summary (n , events, prevalence, t , B)
3. Table 1 with effect sizes and BH-FDR
4. Discrimination: per-subgroup table, gap table with Cohen's h , forest plot, ROC curves with bands
5. Calibration: table, gap table, LOESS curves, Hosmer–Lemeshow with the per-group O vs E blocks
6. Clinical utility: gap table, decision curves with per-subgroup treat-all references
7. Mitigation before/after — *calibration first*, discrimination labelled a sanity check
8. Intersectional cell diagnostics (n , events, shrinkage weight)
9. Appendix: TRIPOD+AI item coverage
10. Appendix: how to read this report

The TRIPOD+AI appendix — the reporting checklist for prediction models built with machine learning — marks items the package addresses and items *you* must supply. It is a checklist, not a certificate.

Sparse intersectional cells

Two-way cells (sex × reserve, age × reserve) go sparse fast. Cells below `intersectional_min_events` have their rate shrunk toward the parent marginal rate:

$$\tilde{r}_c = (1 - w_c) r_c + w_c r_0, \quad w_c = \frac{\tau}{n_c^+ + \tau}$$

In words. A tiny cell's rate is pulled toward the overall rate, by an amount set by how few events it has: $w_c = 0$ is no shrinkage, $w_c = 1$ is full pooling. τ is estimated from the variance across cells.

- Only **rate** metrics are shrunk — AUROC and AUPRC are left alone.
- Report w_c per cell (`cell_diagnostics()`), so the pooling is explicit rather than hidden. Shrunk rows are flagged `shrunk=True` and their discrimination intervals withheld.

Trap. Reading a shrunk cell as an estimate of that cell. It is deliberately pulled toward the margin.

The whole audit, end to end

```
import numpy as np, pandas as pd
from isitfair import FairnessAudit

audit = FairnessAudit(
    y_true=y, y_proba=p_oof,
    sensitive_features={"sex": sex, "age_group": age, "reserve": reserve,
                       ("sex", "reserve"): None},
    threshold=0.10, n_bootstrap=2000, random_state=42,
    resample_scheme="patient", nonestimable_threshold=0.10,
    event_floors={"auroc": 25, "calibration_intercept": 25,          # [v0.4]
                  "calibration_slope": 100, "net_benefit": 50},
)

results = audit.run_all()      # discrimination, calibration, hosmer_lemeshow,
                              # decision_curve, roc_curves,          [v0.4]
                              # calibration_curves, estimability     [v0.4]

est = audit.estimability()     # [v0.4]
for name, df in results.items():
    df.to_csv(f"audit_{name}.csv", index=False)
est.to_csv("audit_estimability.csv", index=False)  # downstream code READS this

audit.report(out="audit_report.html", title="Deployment-readiness audit")
```

Optional: comparing two periods

```
from isitfair import compare_periods                                # [v0.4]

dev = FairnessAudit(y_dev, p_dev, attrs_dev, threshold=0.10, **cfg)
hold = FairnessAudit(y_hold, p_hold, attrs_hold, threshold=0.10, **cfg)

by_period = compare_periods({"2018-2020": dev, "2021-2022": hold})
```

The package will not refit your model — deliberately

Models are your input; this toolkit evaluates predictions. Refitting on the development period and scoring the holdout is your pipeline's job, and it must fit *everything* — preprocessing, feature selection, any resampling — on development data only.

Trap. Reading a period difference as model drift. Documentation practices, coding completeness and outcome ascertainment also change over time. Compare case mix before you attribute anything to the model.

Prespecify these, before you look at anything

1. The **subgroup axes**, and which are patient subgroups vs case-mix reference axes.
2. The **primary axis** for your use case — and therefore what counts as a finding.
3. The **operating threshold**, and the band you will sweep around it.
4. The **event floors**, with a written rationale.
5. The **test families** for FDR correction.
6. That predictions are **out-of-fold**, and that any recalibration will be **cross-fitted**.

Everything on this list is cheap to decide beforehand and impossible to defend afterwards.

What a finished audit says

Not “*the model is fair*” — that claim is not available from any of these numbers. Instead:

- Within-group performance on all three axes, with intervals where the data support them and an explicit *not estimable* where they do not.
- Gaps, with effect sizes, read against case mix and prevalence.
- A named primary criterion, and what it says about deploying *this* score at *this* threshold.
- What mitigation was tried, in what order, and what you *declined* to do and why.
- What remains unresolved, and what evidence would resolve it.

Strong average discrimination and reassuring marginal metrics are not evidence of deployment readiness across subgroups.

Getting it

Install

```
pip install isitfair
```

Python $\geq$ 3.10. Everything in this deck unmarked runs on **0.3.0**; slides marked **v0.4** need **0.4**.

Worked examples that ship with the package

`examples/support2_mortality_audit.py` — a real model on the public UCI SUPPORT2 cohort, including the train/calibration/test split that leak-free mitigation requires. Start there.

Source

github.com/HugoGuillen/isitfair

Questions — and bring your own model.